© 2026 UC Berkeley. All rights reserved.

CS 170 Midterm 2

Write in the following boxes clearly and then double check.

Name : Adithya Giri

SID : 3041015344

Exam Room :
- ○ Dwinelle 155
- ● Dwinelle 145
- ○ Other (Specify):

Name of student to your left : Bhargav Eranki

Name of student to your right : Aarush Khare

- **The exam will last 110 minutes.**
- The exam has 4 questions, each worth 25 points.
- Answer each question in the box provided. If you need more space, continue on any scratch page in the exam and clearly state that you are doing so in the box.
- Good luck!

07117

1 [25 points] Running Minimum

Given array $A = [a_1, a_2, \ldots, a_n]$, we wish to compute the array of *running minimums* $M = [m_1, m_2, \ldots, m_n]$, where each m_i is defined as the minimum value seen in the prefix of A up to index i:

$$m_i = \min\{a_1, a_2, \ldots, a_i\} \quad \text{for } 1 \le i \le n$$

(a) Describe an efficient parallel algorithm to compute M. The total work of your algorithm should be $O(n)$ and the span $O(\log n)$. Assume that comparing any two values takes $O(1)$ time. **Note that if your answer relies on an algorithm from lecture or notes, please describe in a couple of sentences how the algorithm works.**

To compute the running minimums of the given Array A, we use the parallel prefix scan algorithm. This algorithm computes a parallel prefix of the give operation and works as long as the operation is associative.
To compute the parallel prefix scan for values $a_1, a_2, a_3, \ldots, a_n$, first compute $z_k = \min(a_{2k}, a_{2k+1})$ for all $k \in \text{range}(n/2)$. $y_k = z_1 + z_2 + \ldots z_k$, and is computed recursively. y_i is the value of m_{2i}, while $\min(y_i, m_{2i+1})$ is the value of m_{2i+1}. Hence we can get the values of M.
Runtime Analysis:
Computing z_k takes $O(1)$ span and $O(n)$ work
Computing y_i takes $O(\log n)$ span an $O(n)$ work,
since the span has recurrence relation $T(n) \le 1T(\frac{n}{2}) + O(1)$
while the work has recurrence relation $T(n) \le 1T(\frac{n}{2}) + O(n)$
so the work is $O(n)$ and " span is $O(1)$.
Finding M from y_k takes $O(1)$ span and $O(n)$ work,
so the total work is $O(n)$ and the total span is $O(\log n)$
T_1 = work = $O(n)$
T_∞ = span = $O(1)$.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

(b) Prove the correctness of your algorithm. **Note that if your answer relies on an algorithm from lecture or notes, your correctness proof must argue that any crucial conditions necessary to invoke that algorithm are satisfied.**

The algorithm uses the parallel prefix scan algorith. The condition for correctness for this algorithm is that the operation must be associative ie $a \circ (b \circ c) = (a \circ b) \circ c$. This is true for the minimum operation. $Min(a, Min(b,c)) = Min(Min(a,b), c)$. Hence the parallel prefix scan produces the correct output.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

2 [25 points] Matching Subarrays

(a) [15 points] You are given a length m array A and a length n array B where $m < n$, with each entry 1 or -1. Give a $O(n \log n)$ time algorithm to find all locations i such that the length-m subarray of B starting at i (and ending at $i + m - 1$) equals A.

For example, if

$$A = [1, -1, -1] \quad \text{and} \quad B = [-1, 1, -1, -1, -1, 1, -1, -1],$$

then your algorithm should return $\{2, 6\}$.

Construct a polynomial $a(x)$ such that $a(x) = a_1 x^{m-1} + a_2 x^{m-2} + a_3 x^{m-3} \dots a_n x^0$, for $A = [a_1, a_2, \dots a_n]$. Construct a polynomial $b(x)$ such that $b(x) = b_1 x^0 + b_2 x^1 + b_3 x^2 + \dots b_n x^{n-1}$. Then multiply $a(x)$ and $b(x)$ together using the fast fourier transform. For all x^i, where $i \geq m$, if the coefficient at x^i is m, then add $(i-m+1)$ to your output. The final answer is all $(i-m+1)$ such that x^{i-m+1} has a coefficient of m in $a(x)b(x)$.

x^i in $a(x)b(x)$

Justify the correctness of your algorithm in a couple of sentences.

By taking the polynomial multiplication of $a(x)$ and $b(x)$ we are computing the convolution of their coefficients, which can be thought of as a sliding dot product of the coefficients. Since the coefficients of $a(x)$ are $a_1, a_2, \dots a_m$ and the coefficients of $b(x)$ are $b_1, b_2, \dots b_n$, by when the sliding dot product is m, $a_i \times b_{i+m-1}$, so $a_i = b_{i+m}$ for all m.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

(b) [10 points] Give an $O(n \log n)$ time algorithm for the same problem if the entries of the arrays are instead chosen from $\{1, 2, \ldots, k\}$ for some constant k. You may assume $k = 3$ if you like.

Assume $K=3$, Construct a polynomial $a(x) = \frac{1}{a_1}x^{m-1} + \frac{1}{a_2}x^{m-2} + \frac{1}{a_3}x^{m-3} + \ldots \frac{1}{a_m}x^0$. Construct a polynomial $b(x) = b_1x^0 + b_2x^1 + \ldots b_nx^{n-1}$. Multiply $a(x)$ and $b(x)$ together using the Fast Fourier Transform. For all x^i such that $i \geq m$ in $a(x)b(x)$ such that the coefficient of x^i is m add $(i+m-1)$ to your output. The final output is $(i+m-1)$ for all x^{i+m-1} such that the coefficient of x^{i+m-1} is m.

Proof of Correctness:

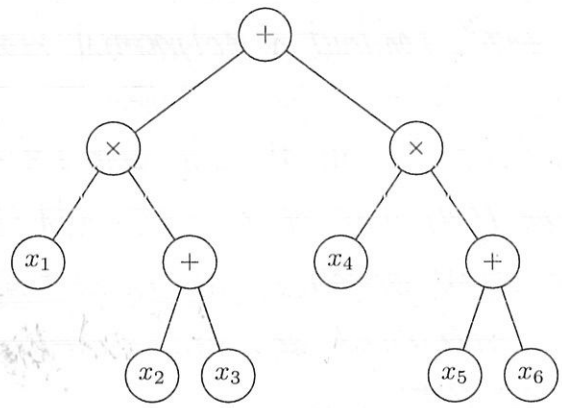
By multiplying $a(x)$ and $b(x)$ we are taking the convolution of their coefficients, which can be thought of as a sliding dot product. If $\frac{1}{a_i} \times b_{m+i} = 1$, $a_i = b_{m+i}$. If the sum of all m $\frac{1}{a_i} \times b_{m+i}$ is m, then $\frac{1}{a_i} \times b_{m+i} = 1$ for all $i \leq m$. This is because we assume $K=3$, so ~~no~~ 1, 2, and 3 are relatively prime so no other sum will equal to m. Hence if $\frac{1}{A} \cdot B[i:m+i] = m$, $A = B[i:m+i]$ hence $(i+m-1)$ should be in our result.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

3 [25 points] Arithmetic Trees

We say an *arithmetic tree* is a binary tree where each leaf is assigned an input variable x_ℓ, and each internal node is assigned an operator ($+$ or $\times$). An example is drawn below:



For each internal node v, we let x_v be the value given by applying the operator at v to the values of its children. Let x_{root} be the value of the root node.

Assume we are given an n-vertex arithmetic tree, along with some assigned value x_ℓ to each leaf ℓ.

(a) [5 points] Give a $O(n)$-time algorithm to compute the values x_v for all the internal nodes v.

Since the arithmetic tree is a tree, it contains no cycles and can be topologically sorted. The runtime of topo sort is $O(n+E)$, and since the arithmetic tree is a tree, $E=n-1$, so topo sort take $O(n)$ time.
We First topologically sort the arithmetic tree, and fill in values for x_i $\forall i \in n$. Then we set the value of a parent node next in the topo sort by applying its operation to its two children. These children are guaranteed to be computed since we have topo sorted the graph.
We can continue this process until we reach the end of the graph.
Runtime Analysis:
Topo sort: $O(n)$ time
Evaluation: Each operation takes $O(1)$ time and we have at most n operations, since its a binary tree.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

(b) [20 points] For nodes u, v we define the *sensitivity* $S_{u,v}$ to be the change in x_v when x_u is increased by 1. These sensitivities satisfy the following *chain rule*: for a path $v_0 \to v_1 \to \cdots \to v_k$, then

$$S_{v_0,v_k} = S_{v_0,v_1} \cdot S_{v_1,v_2} \cdots S_{v_{k-1},v_k}.$$

Using this fact, give a $O(n)$-time algorithm to compute the sensitivities $S_{x_\ell,x_{\text{root}}}$ for all the leaves ℓ.

We topologically sort the tree and reverse the ordering of the sort. We can now compute $S_{v_{k-1}}$ as the product of $S_{v_k,v_{root}} \cdot S_{v_k,v_{k-1}}$. S_{v_k} is guaranteed to be computed since we are traversing the topo sorted tree backwards. We start by computing $S_{v_{root},v_{root}} = 1$, and the calculate the S for the previous nodes and so on.

Runtime Analysis:

Topo Sorting and Reversing takes $O(n)$ time.

Computing $S_{v_{k-1}}$ takes $O(1)$ time, and the are n nodes to compute for, so evaluation take $O(n)$ time.

Briefly justify the correctness of your algorithm.

$S_{v_{k-1},v_{root}} = S_{v_k,v_{root}} \cdot S_{v_{k-1},v_k}$ since $S_{v_k,v_{root}} = S_{v_k,v_{k+1}} \cdot S_{v_{k+1},v_{k+2}} \cdots S_{v_{root-1},v_{root}}$. Hence by storing the value of $S_{v_k,v_{root}}$, we can compute $S_{v_{k-1},v_{root}}$ in constant time. Since we have reverse topo sorted the graph, $S_{v_k,v_{root}}$ is guaranteed to be filled in when computing $S_{v_{k-1}}$.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

4 [25 points] Job Scheduling

We have a set of n tasks that we need to finish as soon as possible (i.e. we want to minimize T: all tasks are completed by time T). Each task $i = 1, \ldots, n$ takes a known amount of time $t_i > 0$ to complete. We can run any number of tasks simultaneously. However, there are dependencies between the tasks: we are given a set of directed edges E, where a directed edge $(i, j) \in E$ means that task i must finish before task j can start.

(a) [7 points] Formulate a linear program (LP) that finds the minimum possible time to complete all n tasks.

Your LP should have a variable $s_i > 0$ for the starting time of each task i. Clearly define the objective function and the constraints on these variables. For convenience, you may assume that $(i, n) \in E$ for every $i < n$, so task n is the last to start.

min s_n

s.t.

for each (u,v) in E

$s_v \geq s_u$

(b) [3 points] Can you see another (faster) algorithm that solves this problem without linear programming? Briefly describe the main idea in a couple sentences.

Create a graph with task i as a node and an edge with weight of t_i between task i and task j if j depends on i to be completed. The diameter of this dag, or the longest path to a sink node is the minimum time to finish all tasks.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

(c) [7 points] Now suppose you want to finish all tasks by a given time T. To make this possible, for each task i, you can choose an amount x_i of dollars to pay for extra resources, which will reduce the task's time to $t_i - x_i$. Due to budgetary constraints you can pay at most C dollars for any given task.

Write a new LP to minimize your total cost for this problem.

$\min \sum x_i$

s.t.

$x_i \leq C$

for all tasks j on the path to S_n

$S_n - \sum x_j \leq T$

(d) [1 point] Do you think your algorithm from part (b) might also work in this new setting? A quick yes/no is sufficient — please don't spend more than a minute thinking about this.

○ Yes ● No

(e) [7 points] Write down the dual of your LP from part (a). **Provide a natural interpretation of the dual in the context of scheduling.** (The dual from part (c) is more complicated but also has a similar natural interpretation.)

Hint: Does the dual solution give information about which tasks are bottlenecking the schedule?

$\max \sum y_{uv} S_u$

s.t.

for all $(u,n) \in E$

$y(u,n) \leq S_n$

Maximize the sum of starting times such that the all tasks start before S_n.

© 2025 UC Berkeley. All rights reserved.

Exam continues on next page ⟹

Use this page for scratch work.

Do not tear this page out.

© 2025 UC Berkeley. All rights reserved.

(0 points) Doodle box! Feel free to draw us something! Please also use this box if you would like to give us suggestions, comments, or complaints.

You may also record any ambiguities in the exam and the assumptions you made, as well as possible alternate interpretations. Note that if you submit a regrade request regarding any ambiguities on the exam, we will only consider the request if you have correctly noted an ambiguity in the box below.

Additionally, if you notice anything suspicious, you can also document it in this box (please be specific: i.e. where are you sitting and what did you notice?).

Indexing unclear on Q2

Use the space below for scratch work.

© 2025 UC Berkeley. All rights reserved.

Use this page for scratch work.

Do not tear this page out.

© 2025 UC Berkeley. All rights reserved.